



**AKADEMIA GÓRNICZO-HUTNICZA
IM. STANISŁAWA STASZICA W KRAKOWIE**

Computational Techniques 2026/2027

Lecture No 1

**Przemysław Korohoda, Dr. EEng.
E-mail: korohoda@agh.edu.pl**

UPEL: CT_2026

Lecture agenda

1. Collaboration rules, marks and credits, semester structure, laboratory groups, etc.
2. Main topics.
3. References and supplemental reading.
4. Selected MATLAB features and elements.

**The presentation rate is adjustable.
Questions can be asked immediately.**

The algorithms/methods are presented in a complete, however rather laconic form – this is intentional, to encourage running your own computational experiments to clarify possible doubts and develop expected depth of understanding the emerging topics.

Collaboration rules, assessment, etc.

Ingredients contributing to the grade after the laboratory:

- 1) possible short written quiz at the beginning of the lab. meetings
(typically 2 questions, lasting for about 10 minutes),**
- 2) expected performance during the exercises,**
- 3) exceptionally active participation,**
- 4) submitting homework (within the deadline),**
- 5) final test - a) theory of algorithms and b) computational experiment.**

For the final grade assessment, the weights are typically distributed in the following way:

60% - laboratory;

40% - project. pass mark/grade is obligatory for both

In each case the grade is related to the points according to the general scale:

Points (%)	grade
50-59	3
60-69	3.5
70-79	4
80-89	4.5
90-100	5

Consultations !!! (possible on-line but also at the University – as agreed)

Main goal and assumption

**To provide answers for intriguing questions via
designing and conducting QUICK computational experiments.**

When we might need such approach?

Let us imagine/assume that we learn (from some lecture or a book or just from the internet) about some theory/algorithm/formula/etc. but some details are not clear to us and/or we are not convinced it works and/or we would like to make sure we understand it properly.

... or we are expected to explain some topic to an audience and we need some particular examples that are not available anywhere where we can easily access them and/or use (for example due to the copyright limitations).

... or we would like to provide some computational laboratory to our class, so that our students could consciously perform their own experiments, but without the necessity of becoming the programming experts.

Therefore, we need to arrange (improvise?) some quick computational experiment, to receive the results with reasonable amount of effort and almost immediately, so that we would rather concentrate on the topic than on the programming code.

An example

Algorithm (general description):

**Looking for solution of the quadratic equation,
in particular with Delta-based algorithm.**

Details (which we should give some thoughts to):

Polynomial definition.

Function values vs. argument values.

Numbers - real or complex? (Domain? Function? Coefficients?)

Definition and impact of the polynomial root/roots.

**Graphical presentation of the polynomial itself but also
of its roots.**

What questions may be asked in such context?

What challenges could be identified? What doubts may be raised?

**How can we use the allowed set of computational tools
(here: from MATLAB) to solve the problem?**

Main topics

Investigation and understanding of selected computational techniques, which includes: polynomials, trigonometric functions, matrices and relevant algebra, interpolation, approximation, solving sets of algebraic equations/differential equations, numerical computation of integrals, statistics, selected operators and transforms.

Engineering applications: analysis and modelling of systems (circuit theory, electronic circuits, signal processing, etc.)

Learning/mastering selected Matlab features

The main goal, however, is to exercise on and develop skills related to:

elaboration and realization of computational experiments (in MATLAB) for „any given topic“.

Literature and supplemental reading:

1. **W.H.Press, S.A.Teukolsky, W.T.Vetterling, B.P.Flannery: Numerical recipes in C.** 2nd ed. Cambridge University Press, 1992 (or later).
2. **A.Biran, M.Breiner: Matlab 5 for engineers.** Prentice Hall, 1999.
3. http://nm.mathforcollege.com/topics/primer_dif.html
4. **Mathworks www page**

etc.

All students with their e-mail address within the AGH-University domain are allowed for free access to MATLAB from their own computers

MATLAB = MATrix LABoratory

Beginning:

Late 1970s. - Cleve Moler, University of New Mexico, noticed inefficiency of repeated writing similar programmes in FORTRAN so he suggested MATLAB: interactive, easy input/output, operations with vectors and matrices in one simple step.

Od 1983 – Eng. Jack Little started his commercial project: **MathWorks;**

**1994 – MATLAB 4.2c for Windows 3.1 (math coprocessor required);
MATLAB enters Polish universities;**

2000 – MATLAB 6.0, using JVM (Java Virtual Machine);

Non-commercial (free) alternative packages:

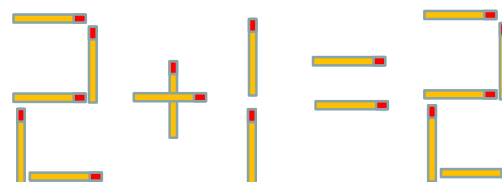
**Scilab (very similar, but differences are noticeable),
GNU octave (required 100% compatibility with MATLAB).**

Alternative commercial packages: Mathematica, Maple, TK Solver, IDL.

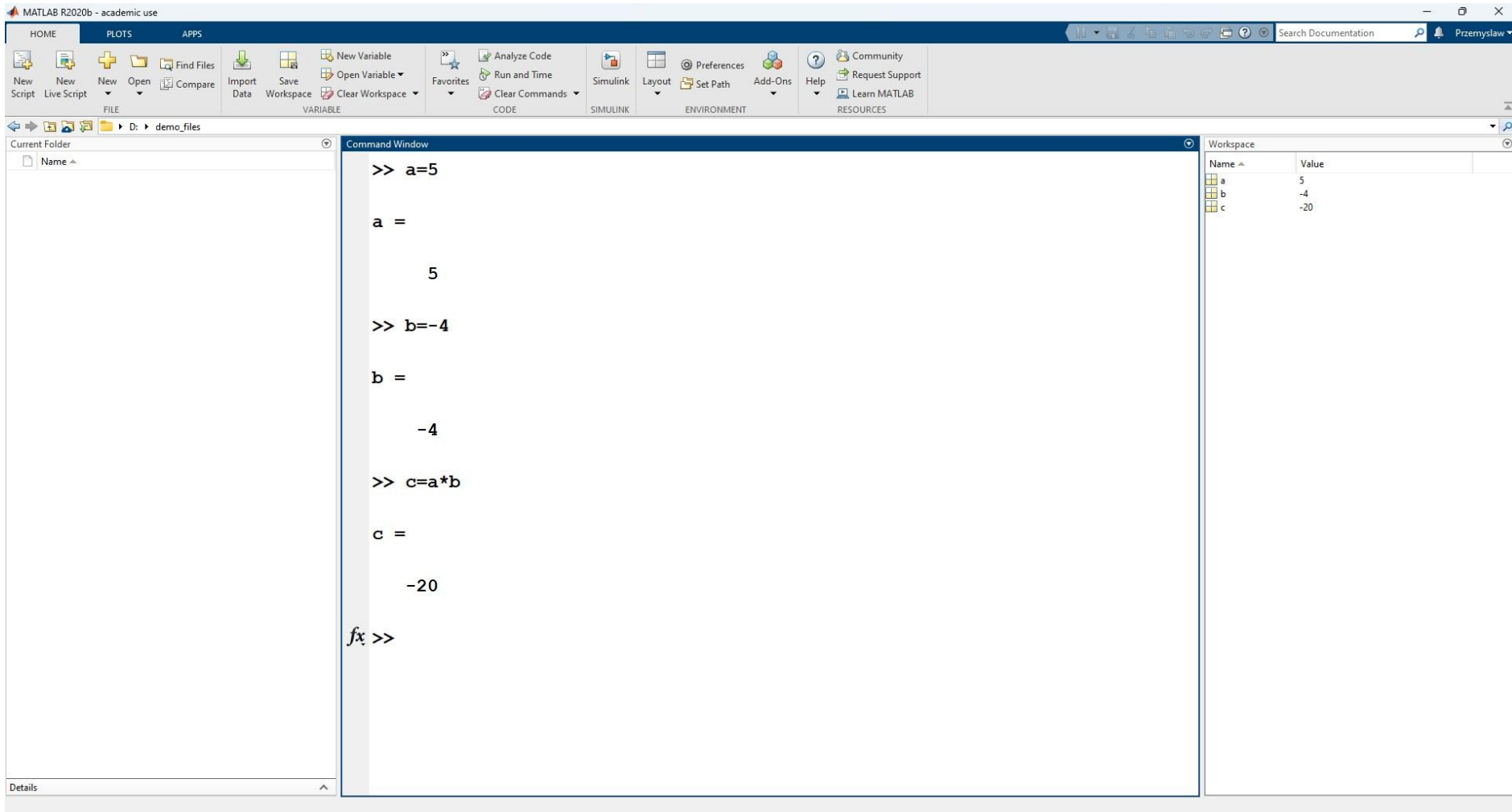
MATLAB + Toolboxes

It is possible:

1. To import software written in C or in FORTRAN (mex-pliki).
 2. To perform symbolic computations (>>help sym).
 3. To use graphical interface (GUI or newer tool) and elaborate simulations in the schematic diagram form (Simulink).
 4. To work with the live scripts.
- etc... **but we are not going to use such "otherwise interesting facilities"**



The main MATLAB windows-set



The image shows the MATLAB R2020b - academic use interface. The top menu bar includes HOME, PLOTS, and APPS. Below the menu bar is a toolbar with various icons for file operations, code execution, and environment management. The main workspace is divided into three panes:

- Current Folder:** Shows the current directory as D:\demo_files.
- Command Window:** Contains the following commands and output:


```
>> a=5
a =
    5
>> b=-4
b =
   -4
>> c=a*b
c =
  -20
fx >>
```
- Workspace:** Displays the current workspace variables:

Name	Value
a	5
b	-4
c	-20

Selected MATLAB elements

Work within the **Command Window** – the commands are executed immediately after we decide so (i.e., when we press the [Enter] key):



```
>> a=5 [Enter]  
a =  
    5  
>>
```

```
>> a=5, [Enter]  
a =  
    5  
>>
```

```
>> a=5; [Enter]  
>>
```

```
>> a=5; [Enter]  
>> b=-4; [Enter]  
>> c=a*b [Enter]  
  
c =  
   -20  
>>
```

>> is „the prompt“

- a, b and c are **scalar** variables
- by default they are in the **double** number format
- * are arithmetic operators

```
>> a=5; [Enter]  
>> b=-4; [Enter]  
>> c=a*b; [Enter]  
>>
```

Nothing could teach you better than your own action

Selected MATLAB elements

Variable name:

1. The name should begin with a letter (upper or lower case).
2. The name cannot contain special symbols.
3. The upper case and lower case specify different names.
4. There are some predefined variables and you should not override them.
5. You should not override the names of built-in functions (in particular those you think you are going to use).

Examples of special symbols:

1. Operators, e.g.: `+` `-` `*` `/` `^` `\` `|` `&` `:`
2. Separators, e.g.: `,` `;` `.` *space*
3. Brackets: `()` `[]` `{}`
4. Anything that may have special meaning for MATLAB ...

Predefined variables:

`pi` – π number (ca. 3.14...)

`i` – imaginary unit

`j` – imaginary unit

`eps` – available precision of numbers

`ans` – default variable for computations output

Selected MATLAB elements

Examples of correct variables names:

A

B_23

c21

tn

time_of_computations_in_minutes

Incorrect names (examples):

1A

– first character is a digit

R%

– comment operator (%) included

T-start

– subtracting operator included

First variable

– 'space' included

To avoid (examples):

cos

- overrides built-in cosine function

i

- overrides built-in imaginary unit

Selected MATLAB elements

In case of incorrect name, MATLAB will tell us about it:

```
>> 1a=5 [Enter]
```

```
??? 1a=5
```

```
|
```

```
Error: Missing operator, comma, or semicolon.
```

but – as you see – the comment may indicate some other source of error...

In case of 'not suggested' names, you will probably learn later that you have used them incorrectly, but at some cost of negative emotions ...

Note – from now on, in the examples, we will omit the [Enter] key information.

Basic arithmetic operators

- +** addition
- subtraction
- *** multiplication
- /** division (right hand)
- ^** power

Note: in MATLAB all of them work in the matrix mode

Numerical variables in MATLAB

The most basic variable type is a **matrix**, which has certain number of rows and columns (in precisely this order), i.e.:

$$A = \begin{bmatrix} 1 & \sqrt{2} \\ 1/3 & -5 \\ -\pi & 0 \end{bmatrix}$$

A particular case could be a **column vector** (i.e. one-column matrix), for example:

$$v = \begin{bmatrix} b \\ -2 \\ \sqrt[3]{-8} \end{bmatrix}$$

...or a **row vector** (i.e. one-row matrix), for example:

$$v = [b \quad -2 \quad \sqrt[3]{-8}]$$

Another special case could be a **scalar number** (a one-row and one column matrix), for example:

$$b = [\sqrt[4]{2}] = \sqrt[4]{2}$$

Numerical variables in MATLAB

Some extension of the matrix concept is an **array** which may be built with use of more than two dimensions, like this one:

$$\mathbf{X}_{3 \times 2, 1} = \begin{bmatrix} 1 & \sqrt{2} \\ 1/3 & -5 \\ -\pi & 0 \end{bmatrix}$$

$$\mathbf{X}_{3 \times 2, 2} = \begin{bmatrix} -1 & \sqrt{-2} \\ 3/2 & 5 \\ \pi & 2.3 \end{bmatrix}$$

$$\mathbf{X} = \mathbf{X}_{3 \times 2 \times 3}$$

$$\mathbf{X}_{3 \times 2, 3} = \begin{bmatrix} 0.4 & \sqrt{3} \\ 3^4 & 555 \\ \sqrt{\pi} & 2.3 \cdot 10^{-5} \end{bmatrix}$$

Scientific notation:

`>> a=2.3e-5`

Note the decimal separator!

Dealing with matrices (an intro...)

Creating a matrix:

```
>>A=[1,2,4;-1,5,-3],  
A =  
     1     2     4  
    -1     5    -3
```

Multiplying matrices:

```
>>A=[1,2 ; -1,-3];  
>>B=[1,0 ; 0,1];  
>>C=A*B,  
C =  
     1     2  
    -1    -3  
  
>>
```

dot-preceded operator

```
>>A=[1,2 ; -1,-3];  
>>B=[1,0 ; 0,1];  
>>C=A.*B,  
C =  
     1     0  
     0    -3  
  
>>
```

A common dilemma: names of the variables

How many would that be: $c = a \cdot b$

when: $a = 5, b = -4$?

```
>> a=5;  
>> b=-4;  
>> c=a*b
```

```
c =  
-20  
>>
```



```
>> first_input_variable = 5;  
>> second_input_variable = -4;  
>> product_result = first_input_variable * second_input_variable
```

```
product_result =  
-20  
>>
```

Some particular MATLAB features

The colon as a series operator (**start:step:stop** or **start:stop**):

```
>>a=-1:0.5:2,
a =
    -1.0000    -0.5000         0     0.5000     1.0000     1.5000     2.0000
>>
```

four digits displayed by default...

Making the vector vertical **x=x(:);**

```
>>x=1.1:4,
x =
    1.1000    2.1000    3.1000
>>x=x(:),
    1.1000
    2.1000
    3.1000
>>x=x',
x =
    1.1000    2.1000    3.1000
```

Round brackets immediately next to the variable name make the field for indexes (indices):

```
>>x=2:7; y1=x(3),
y1 =
     4
>>y2=x(2:2:5),
y2 =
     3     5
```

The indexing always starts with 1

The transposition operator (at this moment for real matrices only)

MATLAB features (cont.)

Matrix indexing:

```
>>x=A(2,3),  
x =  
    -3
```

The **end** keyword:

```
>>x=2:7; y=x(end-2:end-1),  
y =  
     5     6  
>>
```

An example:

```
>>A=[1,2,4,-1 ; -1,5,-3,2 ; 3,2,-7,1],  
A =  
     1     2     4    -1  
    -1     5    -3     2  
     3     2    -7     1  
>>B=A(2:3,1:2:4),  
B =  
    -1    -3  
     3    -7  
>>
```

Note – both sides of the assignment must be in the exact match

Let us denote matrices with the first symbol being capital, like: A, A1, A_in, D_prod, etc.

Round brackets as the priority control tool

In the arithmetic expressions:

```
>>x=2+1*3,
x =
      5
>>x=(2+1)*3,
x =
      9
```

```
>>x=2+2/2,
x =
      3
>>x=(2+2)/2,
x =
      2
```

```
>>x=2.^0:5,
x =
      1      2      3      4      5
>>x=2.^(0:5),
x =
      1      2      4      8     16     32
```

Note - an exception!

Some basic programming tools

Calling the built-in functions:

```
>>A=rand(2,4),
```

```
A =
```

```
    0.8214    0.6154    0.9218    0.1763  
    0.4447    0.7919    0.7382    0.4057
```

```
>>
```

Returns the pseudo-random numbers
taken from the interval [0,1],
with the uniform PDF

```
>>t=0:1e-2:1; x_t=4*cos(2*pi*3*t);
```

$$x(t) = 4 \cdot \cos(2 \cdot \pi \cdot 3 \cdot t)$$

```
>>format long
```

```
>>A=rand(2,4),
```

```
A =
```

```
    0.814723686393179    0.126986816293506    0.632359246225410    0.278498218867048  
    0.905791937075619    0.913375856139019    0.097540404999410    0.546881519204984
```

```
>>format short
```

```
>>help format
```


Script m-files

A simple program:

```
>> a=5;
>> b=-4;
>> c=a*b
```

```
c =
    -20
```

```
>>
```



```
>> a=5; b=-4; c=a*b
c =
    -20
```

```
>>
```

Browsing through the command history... (^/v)

Script m-file:

Minimum version (task_1.m):

```
a=5;
b=-4;
c=a*b
```

Execution:

```
>> task_1
c =
    -20
>>
```

Our **project** version:

```
% The file name
% Description
% Author, date

clc; clear; close all;

% input data:
a=5; b=-4;

        c=a*b, % result

% EoF
```

clc
clear
close
help

**Note – take care of
your file placement !!!**

Your own functions

For the beginning – in the separate m-files (here: my_function_1.m)

```
% The file name
% Description
% Author, date

clc; clear; close all;

% input data:
a=5; b=-4;

    c=a*b, % result

% EoF
```



```
function c=my_function_1(a,b)
% c=my_function_1(a,b)
%
% Description
%
% Author, date

    c=a*b, % result

% EoF
```

Execution:

```
>> my_function_1(5,-4)
ans=
    -20
>>
```

Execution when there are more (than one) returned variables:

```
>> [c,d] = my_function_1a(a,b)
>> c = my_function_1a(a,b)
```

or when we need only the second one:

```
>> [~,quotient] = my_function_1a(5,-4)
```

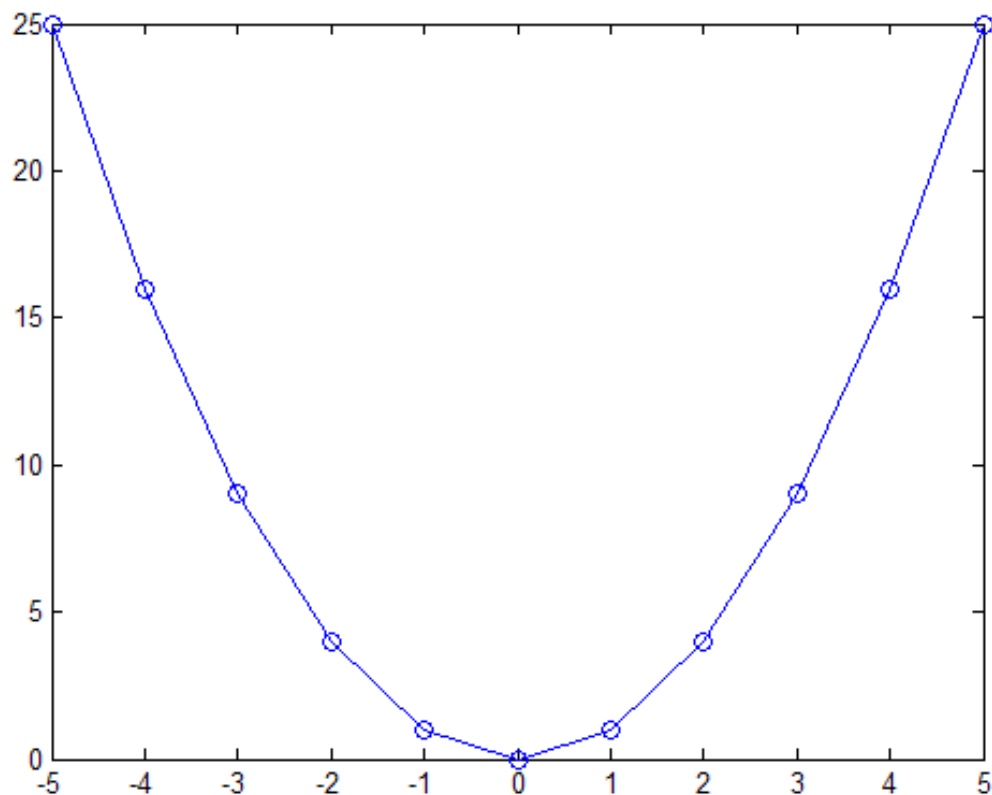
**Note – take care of
your file placement !!!**

Basic graphs (functions: **figure**, **plot**, **clf**)

An example:

```
% .....  
  
clf; clear; close all;  
  
x = -5:1:5;  
y = x.^2;  
  
figure(1); clf;  
plot(x,y,'bo- ');  
  
% EoF
```

*text constant
(a character matrix)*

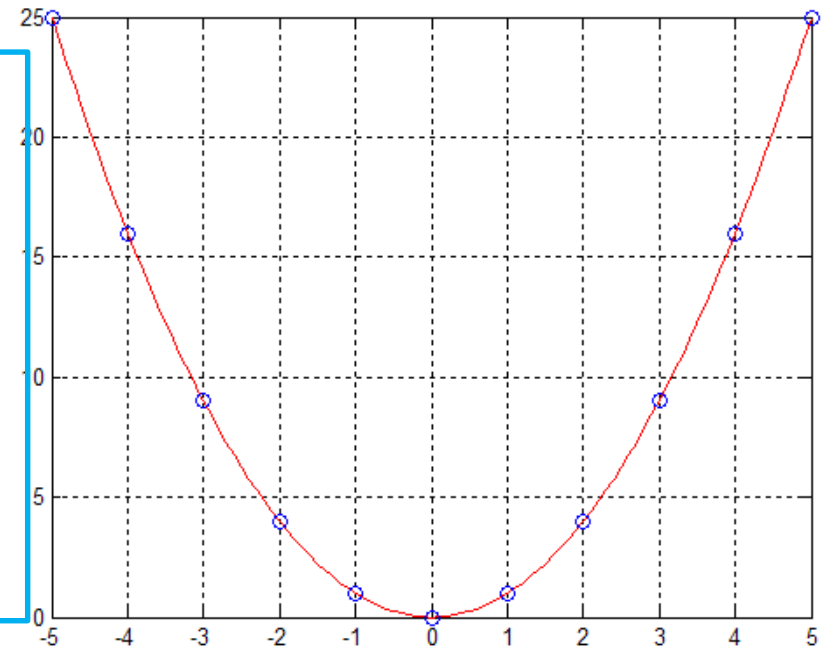


For more options try: >> help plot

The function may not return anything – it could just create a graphical window or plot something

Basic graphs (cont.)

```
% .....  
  
clc; clear; close all;  
  
x = -5:0.1:5;  
y = x.^2;  
  
figure(1); clf;  
    plot(x,y, 'r- ');    grid on;    hold on;  
    plot(x(1:10:end),y(1:10:end),'bo');  
  
% EoF
```



Selected functions related to graphical windows

text const.

title – description above the graph, e.g.: **title('y = y(x)')**

Note the special meaning of „_” or „^”

xlabel – description below the horizontal axis, e.g.: **xlabel('x')**

ylabel – description next to the vertical axis, e.g.: **ylabel('y')**

axis – setting the ranges for both axes, e.g.: **axis([x_start,x_stop,y_start,y_stop])**

A four-element horizontal vector

Let us continue in the lab ...